

Ejercicio 3 - Sistema de Control de Órdenes de Servicio

1. Clases y propósito

OrdenServicio: Representa una orden individual y encapsula sus cinco datos.

GestorOrdenes: Administra la colección dinámica y concentra las reglas de registro, consulta, búsqueda, modificación, cancelación y cálculos.

OrdenNoEncontradaException: Excepción comprobada que comunica que una operación requiere una orden inexistente.

Main (driver program): Muestra el menú, lee datos, llama al gestor y controla excepciones sin terminar inesperadamente.

2. Tablas de propiedades

No.	Tipo	Nombre	Visibilidad	Propósito	Clase
1	List<OrdenServicio>	ordenes	private final	Colección dinámica de órdenes activas. Se declara con la interfaz List y se instancia con ArrayList.	GestorOrdenes
2	int	numeroOrden	private	Identificador único y positivo de la orden.	OrdenServicio
3	String	nombrePropietario	private	Nombre no vacío del propietario.	OrdenServicio
4	String	placaVehiculo	private	Placa no vacía utilizada también para búsquedas.	OrdenServicio
5	String	descripcionServicio	private	Trabajo solicitado; puede modificarse.	OrdenServicio

6	double	costoEstimado	private	Costo positivo; puede modificarse y se usa en los reportes.	OrdenServicio
7	Scanner	TECLADO	private static final	Entrada compartida del programa.	Main
8	GestorOrdenes	GESTOR	private static final	Gestor compartido por las opciones del menú.	Main

3. Tablas de métodos

No.	Método	Visibilidad	Parámetros y retorno	Propósito / excepción
1	OrdenServicio	public	OrdenServicio(int numeroOrden, String nombrePropietario, String placaVehiculo, String descripcionServicio, double costoEstimado)	Inicializa una orden con los datos ingresados.
2	getters	public	getNumeroOrden(): int; getNombrePropietario(): String; getPlacaVehiculo(): String; getDescripcionServicio(): String; getCostoEstimado(): double	Devuelven los valores encapsulados.
3	setDescripcionServicio	public	(String descripcionServicio): void	Actualiza la descripción.
4	setCostoEstimado	public	(double costoEstimado): void	Actualiza el costo.
5	toString	public	(): String	Devuelve toda la información formateada.
6	GestorOrdenes	public	GestorOrdenes()	Crea ordenes = new ArrayList<>().

7	registrarOrden	public	(OrdenServicio orden): void	Valida y agrega; lanza IllegalArgumentException ante datos inválidos o duplicados.
8	validarDatos	private	(int numeroOrden, String propietario, String placa, String descripcion, double costo): void	Aplica las validaciones mínimas.
9	existeNumeroOrden	private	(int numeroOrden): boolean	Indica si el número ya está registrado.
10	buscarOrden	public	(int numeroOrden): OrdenServicio	Busca por identificador; lanza OrdenNoEncontradaException.
11	modificarOrden	public	(int numeroOrden, String nuevaDescripcion, double nuevoCosto): void	Busca, valida y modifica descripción/costo.
12	cancelarOrden	public	(int numeroOrden): void	Busca y elimina la orden de la colección.
13	obtenerOrdenes	public	(): List<OrdenServicio>	Devuelve una copia de las órdenes activas.
14	buscarPorPlaca	public	(String placa): List<OrdenServicio>	Recorre la colección y devuelve todas las coincidencias.
15	calcularTotal	public	(): double	Suma los costos con un for-each.
16	calcularPromedio	public	(): double	Divide el total entre size; devuelve 0 si no hay órdenes.
17	obtenerOrdenMayorCosto	public	(): OrdenServicio	Recorre la lista conservando la orden de mayor costo.
18	obtenerCantidadOrdenes	public	(): int	Devuelve ordenes.size().

Métodos de la excepción

No.	Método	Visibilidad	Parámetros y retorno	Propósito / excepción
1	main	public static	(String[] args): void	Ejecuta el menú hasta elegir 10.
2	mostrarMenu	private static	(): void	Imprime las diez opciones.

3	leerEntero	private static	(String mensaje): int	Reintenta ante NumberFormatException.
4	leerDouble	private static	(String mensaje): double	Reintenta ante NumberFormatException.
5	leerTexto	private static	(String mensaje): String	Lee y recorta texto.
6	registrarOrden	private static	(): void	Solicita datos y captura IllegalArgumentException.
7	consultarOrdenes	private static	(): void	Muestra todas las órdenes.
8	buscarOrden	private static	(): void	Captura OrdenNoEncontradaException y siempre ejecuta finally.
9	modificarOrden	private static	(): void	Captura orden inexistente o datos inválidos.
10	cancelarOrden	private static	(): void	Captura orden inexistente.
11	consultarPorPlaca	private static	(): void	Solicita placa y muestra coincidencias.
12	mostrarReporteCostos	private static	(): void	Muestra total y promedio.
13	mostrarOrdenMayorCosto	private static	(): void	Muestra la de mayor costo o controla lista vacía.
14	mostrarLista	private static	(List<OrdenServicio> ordenes, String mensajeVacio): void	Reutiliza el recorrido e informa si no hay datos.
15	OrdenNoEncontradaException	public	(String mensaje)	Construye la excepción con un mensaje explicativo.

4. Respuestas del análisis

- Información de la colección: Objetos OrdenServicio completos. Solo permanecen las órdenes activas;cancelar elimina el objeto.
- List y ArrayList: El atributo se declara List<OrdenServicio> para programar contra la interfaz y elconstructor crea new ArrayList<>() como implementación dinámica.
- Valores iniciales: OrdenServicio recibe sus cinco valores mediante constructor. GestorOrdenes inicia con una lista vacía. Main crea un Scanner y un GestorOrdenes una sola vez.

9. Identificación: El número de orden es la clave única. Se recorre la lista y se compara getNumeroOrden().
10. Agregar/eliminar: Se usa ordenes.add(orden) después de validar; para cancelar se busca el objeto y se usa ordenes.remove(orden).
11. Búsqueda por placa: Un for-each compara las placas con equalsIgnoreCase y agrega cada coincidencia a otra List.
12. Total/promedio: Un for-each acumula costos. El promedio es total/size; si la lista está vacía se devuelve 0 para evitar división entre cero.
13. Mayor costo: Se toma el primer elemento como candidato y se sustituye cuando aparece un costo mayor. Si no hay órdenes se lanza la excepción propia.
14. Posibles excepciones: Texto no numérico, datos vacíos, costo o número no positivo, número duplicado, orden inexistente y lista vacía al pedir el mayor costo.
15. Operaciones protegidas: Lectura de enteros/costos, registro, búsqueda, modificación, cancelación, consulta por placa y obtención de la orden más costosa.
16. finally: La búsqueda imprime siempre que el proceso terminó y que se regresa al menú, exista o no la orden. finally es apropiado porque esa acción debe ejecutarse en ambos resultados.

5. Diseño UML

